Experiment 1: Nacho Threads

Nachos is a good **instructional** Operating System for teaching basic operating systems.

Nachos Overview

- Nachos kernel and hardware simulator run together in the same UNIX process.
- Differences from earlier systems:
  1. The simulation is deterministic. Debugging non-repeatable execution sequences is a fact of life for professional operating systems engineers.
  2. Instead of using UNIX signals to simulate asynchronous devices such as the disk and the timer, Nachos **maintains a simulated time that is incremented whenever a user program executes an instruction** and whenever a call is made to certain low-level operating system routines. Interrupt handlers are then invoked when the simulated time reaches the appropriate point.
  3. Nachos is implemented in a subset of C++. Object-oriented programming is becoming more popular. It was a natural idiom for stressing the importance of modularity and clean interfaces in building operating systems. To simplify matters, certain aspects of the C++ language are omitted: derived classes, operator and function overloading, and C++ streams.

Scheduler

- Maintains a list of threads that are ready to run: ready list.
- Scheduler is invoked whenever the current thread gives up the CPU (non-preemptive)
- Simple scheduling policy is used.
  1. Assume equal priority for all threads.
  2. Select threads in FCFS fashion.
  3. Append at the end and remove from the front.
  4. The scheduler code can be found in threads/scheduler.cc.
  5. ReadyToRun () : makes thread ready to run, adds it to ready list but does not start the thread yet
  6. FindNextToRun() : thread simply returns the thread at the front of the ready list
  7. Run(thread): switches from one thread to another
     - Check whether the current thread overflowed its stack.
     - Change the state of the new thread to running.
     - Perform the actual context switch (by calling Switch ())

- Terminate previous thread (if applicable)

Thread Management: Concurrency

- Different states it can be in:
  1. READY: eligible to run
  2. RUNNING: only one thread can be in this state
  3. BLOCKED: waiting for some external event
  4. JUST_CREATED: temporary state used during creation
- For simplicity, thread scheduling is normally non-pre-emptive, but to **emphasize the importance of critical sections**, there is a command-line option that causes threads to be time-sliced at "random", but repeatable, points in the program. Concurrent programs are correct only if they work when "a context switch can happen at any time".
- This lab's thread system implements **thread fork**, **thread completion**, along with **semaphores** for synchronisation.
- Implementation can be found in threads/thread.cc
- Fork(function,arg,flag)
  1. function: procedure to be executed concurrently
  2. arg: single parameter which is passed to the procedure
  3. join: a flag which indicated whether the thread can be joined , not important
  4. Turns a thread into one that can be executed
  5. Calls readyToRun()
- Yield()
  1. Finds the next thread to run using findNextToRun()
  2. If another thread has been found, call readyToRun() for old thread and run the new thread using Run()
- Sleep()
  1. Set Status to BLOCKED
  2. Find another thread to run using findNextToRun()
  3. If another thread has been found, run the new thread using Run()
- Finish()
  1. Called at the end of execution
  2. Marks a thread for termination

Lab Implementation

threadtest.cc

- ThreadTest()-test procedure called from within main
  1. Thread *t1 = new Thread("child1"); minimal initialisation using constructor
- SimpleThread(_int which)-
  1. Which tells u which thread it is
  2. currentThread->Yield(); causes current thread to give up on the CPU
  3. each thread is executing this function

Experiment 2: CPU Scheduling in NachOS

Non-preemptive FIFO Scheduling

- Thread::Fork() invokes Scheduler::readyToRun(). Appends new thread at the end of the readyList
- Thread::Yield(),Sleep(),Finish() invoke Scheduler:: Find NextToRun(). Selects next thread to run (head of readyList) and executes it using Scheduler::Run().
- Thread::Yield() invokes Scheduler::ReadyToRun(). Adds thread back at the end of the readyList

Threads

- Thread()
  1. Constructor : sets the thread as JUST_CREATED status
- Fork()
  1. Allocate stack, initialize registers
  2. Call Scheduler::ReadyToRun() to put the thread into readyList, and set it status as READY
- Yield()
  1. Suspend the calling thread by Scheduler::Run() which sets its status as RUNNING and call SWITCH() (in code/threads/switch.s) to exchange the running thread
- Finish()
  1. Mark current thread for destruction
  2. Call Sleep() to find next thread to run and execute it
- void Yield()
  1. suspend the calling thread and select a new one for execution

2. Find next ready thread by calling Scheduler::Find NextToRun()
3. Put current thread into ready list(waiting for rescheduling)
4. Execute the next ready thread by invoking Scheduler:Run().
5. If no other threads are ready to execute, continue running the current thread

- Void Finish()
  1. Terminate the currently running thread
  2. Call Sleep() and never wake up
  3. Deallocate the data structures of a terminated thread
  4. The newly scheduled thread examines the toBeDestroyed(terminated) variable and finishes this thread
- Void Sleep()
  1. Suspend the current thread and change its state to BLOCKED
  2. Run next ready thread
  3. Invoke interrupt->Idle() to wait for the next interrupt when readyList is empty
  4. Sleep is called when the current thread needs to be blocked until some future event takes place
  5. E.g Waiting for a disk read interrupt
  6. It is called by Semaphore::P() in code/threads/synch.cc
  7. Semaphore::V() will wake up one of the thread in the waiting queue(sleeping threads queue)

Timers

- Used to trigger an interrupt (after a fixed number of time ticks)
- void TimerInterrupthandler() : interrupt handler that is called when timer expires
  1. function defined in code/threads/system.cc
  2. executes whenever the associated timer expires and the interrupt is triggered
  3. timer is initialised in in code/threads/system.cc using the constructor for class Timer which is defined in code/machine/timer.cc
- void TimerExpired() : function that executes when the timer expires
  1. function defined in code/machine/timer.cc
  2. executes whenever the timer expires, in turn invoke the interrupt handler
- int TimeOfNextInterrupt() : function returns the next interrupt time click
  1. defined in code/machine/timer.cc
  2. returns an integer denoting the number of time ticks
  3. used to schedule an interrupt using the timer. Interrupt will be triggered after this number of time ticks from the current time

4. can be used to make the timer periodic as required for round robin scheduling

Interrupt

- Timer uses several functions from the interrupt class
- Pending timer interrupts in the system are maintained in a list called pending, comprising objects of the class PendingInterrupt
- List is sorted in increasing order of time tick when the interrupt will be triggered
- Defined in code/machine/interrupt.cc
- Void Schedule()
  1. Function schedules/inserts a new interrupt to the pending list
  2. Insertion is in sorted order; sorted by the pending time ticks for the interrupt to be triggered
  3. Used in Timer to initialise a timer interrupt
- Void OneTick()
  1. Function to process a single time tick
  2. Updates global variable stats->totalTicks
  3. Calls Interrupt::CheckIfDue() to process any pending interrupt that would be triggered now
  4. If variable yieldOnReturn is true, then triggers a context switch through a call to Thread::Yield()
- Bool CheckIfDue()
  1. Function to process interrupts and invoke handler
  2. Checks if pendingInterrupt at the head of pending list should be triggered at current tick
  3. If yes, corresponding handler is invoked
  4. Handler for Timer is Timer:: TimerExpired()]
- Void YieldOnReturn()
  1. Function that is called by the Timer handler TimerInterruptHandler()
  2. Sets the variable **yieldOnReturn to true**
  3. Force Interrupt::OneTick() to trigger context switch

Lab Quiz 1
(Most are straightforward)
- what is false about NachOS? (it directly runs on hardware)
- what is false about the Schedule()? (List is sorted in descending order)
- what does the join in fork() argument do? (True, the parent call and wait for the child process, a flag which indicated whether thread can be joined)
- what does the FindNextToRun() do?(thread simply returns the thread at the front of the ready list)
- what does the Run() do?(switches from one thread to another)
- What does the TimeofNextInterrupt() do?(function returns the next interrupt time click)
- What best describes the NachOS scheduling policy? (i put none of the above cos NachOS is FCFS)
- What is the output of the following code in csv? (Look at the lab 1 csv how you can copy and paste it)
- In the initalise() function, what trigger the timer()
(Yes, randomYield set to True)
- What is the state when Thread() is used? (Just Created)
Other people question bank
1) what's the time quantum for this code
2) what's the function to edit to make round robin(TimeOfNextInterrupt())
3) does this code show that interrupt timer is implemented (the randomyield=true)
4) data structure to implement ready list (queue)
5) what thread is in the running state at this tick
6) what is the output of this code (lab 1 threadtest)

q1. steps to switching thread
- ans: 4 steps found in slides under Run(thread)

- Check whether the current thread overflowed its stack.
- Change the state of the new thread to running.
- Perform the actual context switch (by calling Switch ())
- Terminate previous thread (if applicable)

q2. the fn FindNextToRun() simply returns the thread at the front of the ready list. true or false?
- ans: true

q3. what is false about nachOS?
- ans: nachOS runs directly on hardware

q4. what is the status of the thread after Thread()?
- ans: JUST_CREATED

q5. if yieldOnReturn is true, function triggers a context switch through a call to Thread::Yield(). true or false?
- ans: true

1. Which of the following allows to add a new interrupt to the pending list? (More than one option question)
- **Schedule()** / OneTick() / Yield() / None of the options

2. What state is a thread put in when it is removed from the waiting state (for example after acquiring a lock)?
- **READY** / RUNNING / WAITING / something i forgot